

# Звіт до проєкту «Конструктор логічних елементів»

## 1 Загальна інформація про проєкт

У цьому проєкті була реалізована симуляція арифметико-логічного пристрою (ALU) та базових цифрових схем за допомогою **Python**. Основна ідея полягала в тому, щоб показати принципи роботи логічних вентилів, суматорів та комп'ютерних компонентів на базовому рівні.

У процесі роботи були створені:

- базові логічні вентиля;
- система передачі сигналів через дроти;
- full adder (повний суматор);
- 4-бітний суматор;
- 4-бітний ALU;
- графічний інтерфейс.

## 2 Структура проєкту

Проєкт має наступну ієрархію файлів:

- `Interface/` — `interface.py` (запуск GUI) та `interface.ui` (файл Qt Designer).
- `dev/` — `alu.py` (реалізація ALU) та `data.py` (базові класи логіки).
- `images/` — набір іконок вентилів, перемикачів та елементів інтерфейсу.
- `main.py` — точка входу до проєкту.

## 3 Використані технології

- **Python** — основна мова програмування.
- **PyQt5** — бібліотека для створення графічного інтерфейсу (вікна, кнопки, взаємодія).
- **Qt Designer** — інструмент для візуального редагування інтерфейсу (`.ui` файл).

## 4 Архітектура та реалізація цифрової логіки

Проєкт побудований на принципах ООП.

## 4.1 Клас Signal

Представляє логічний сигнал: `True` = 1, `False` = 0.

## 4.2 Клас Wire

Моделює провідник. При зміні сигналу автоматично викликається метод `update()` у всіх підключених вентилях, що реалізує механізм поширення сигналу.

## 4.3 Клас Gate

Базовий клас для всіх вентилів. Має методи:

- `evaluate()` — обчислення логічної операції.
- `update()` — оновлення виходу після зміни входу.

## 5 Логічні вентиля та формули

Реалізовані такі елементи: NOT, AND, NAND, OR, NOR. Складніші елементи побудовані на основі базових:

- **XOR**:  $XOR(a, b) = (a \vee b) \wedge \neg(a \wedge b)$
- **XNOR**:  $XNOR(a, b) = (a \wedge b) \vee (\neg a \vee \neg b)$

## 6 Арифметичні компоненти

### 6.1 Мультиплексор (Mux4to1)

Обирає один із 4 входів (`w0-w3`) залежно від керуючих сигналів (`s1`, `s0`).

### 6.2 Full Adder (Повний суматор)

Виконує додавання двох бітів та вхідного переносу (`Cin`):

- $Sum = A \oplus B \oplus C_{in}$
- $C_{out} = (A \wedge B) \vee (B \wedge C_{in}) \vee (A \wedge C_{in})$

### 6.3 4-бітний суматор та ALU

4-бітний суматор реалізований за типом **Ripple Carry Adder**[cite: 203, 211]. **ALU4bit** підтримує операції: ADD, SUB (через доповнення до двійки), AND, NOT.

## 7 Інструкція для запуску

1. Встановити Python 3.
2. Встановити PyQt5: `pip install PyQt5`.
3. Запустити проєкт: `python main.py`.

## 8 Висновок

Було створено функціональну модель ALU з графічним інтерфейсом, що демонструє роботу цифрової логіки від рівня окремих вентилів до складних арифметичних операцій.